

```

def getTimeInHHMMSS(self, nMillisecondsDigits=0): ❸
    --- snip ---
    nSeconds = self.getTime()
    mins, secs = divmod(nSeconds, 60)
    hours, mins = divmod(int(mins), 60)

    if nMillisecondsDigits > 0:
        secondsWidth = nMillisecondsDigits + 3
    else:
        secondsWidth = 2

    if hours > 0:
        output =
            f'{hours:d}:{mins:02d}:{secs:0{secondsWidth}.
{nMillisecondsDigits}f}'
    elif mins > 0:
        output = f'{mins:d}:{secs:0{secondsWidth}.{nMillisecondsDigits}f}'
    else:
        output = f'{secs:.{nMillisecondsDigits}f}'

    return output

def stop(self): ❹
    """Stops the timer"""
    self.getTime() # remembers final self.savedSecondsElapsed
    self.running = False

```

Listing 13-4: The `CountUpTimer` class

The implementation depends on three key instance variables ❶:

- `self.running` is a Boolean that indicates whether the timer is running or not.
- `self.savedSecondsElapsed` is a float that represents the elapsed time of a timer.
- `self.secondsStart` is the time that the timer started running.

The client calls the `start()` method ❷ to start a timer. In response, the method calls `time.time()`, stores the start time in `self.secondsStart`, and sets `self.running` to `True` to indicate that the timer is running.